

LUT-SHA256

SHA-256 resolved through chained look-up tables: theory, security analysis and method compression

Abraham A. — Independent Research

Version 1.2 · August 27, 2026

Contents of this edition

- Theorems 1–5 with formal inductive proofs
- Vulnerability ladder V1.0–V1.7 and leakage thresholds (SMT/Z3)
- Method compression v2.0–v2.4 with verified metrics
- New chapter: applications and use cases (hardware wallets, CIM, FPGA)

Contents

1	Executive summary	2
2	Background: the original SHA-256 (FIPS 180-4)	2
3	The LUT-SHA256 method: theory	2
4	Comparison with the original method	3
5	Experimental verification	3
6	Implementation vulnerabilities	4
7	Vulnerability ladder V1.0–V1.7 and attacks	4
8	Compression of the method v2.0–v2.4	5
9	Applications and use cases	5
9.1	9.1 Hardware wallets and secure elements (BIP39/PBKDF2)	5
9.2	9.2 Compute-in-Memory (CIM)	6
9.3	9.3 FPGAs and low-power ASICs	6
9.4	9.4 Side-channel research and education	6
9.5	9.5 Formal verification and auditing	6
9.6	9.6 Education	6
9.7	9.7 Platform fit and non-applicable scenarios	6
10	Conclusions and future work	7
11	Annex A — Complete Exercise A (entropy 7f x 16, BIP39)	7
12	Annex B — Complete Exercise B (00 x 64, two blocks)	8
13	Annex C — Reference code (v2.4 engine, fragment)	9
14	Annex D — Complete constants	9

1. Executive summary

This document presents LUT-SHA256, an alternative method for executing the SHA-256 hash function in which no arithmetic, logical or shift operation is ever applied to message data: bytes act only as indices into precomputed tables. The central idea is simple — every finite function is, at bottom, a table — but its practical realization required solving two structural problems: addition modulo 2^{32} , reformulated as a chain of tabulated byte carries (TADDC), and rotations, converted into byte recombination through displacement tables. The method is formally equivalent to FIPS 180-4 (Theorem 5) and produces identical digests in every verified scenario: official vectors (“abc, empty”), the BIP39 exercises of the study (entropies *7times16* and *00imes64*), critical padding boundaries (55/56/57 bytes), long messages and zero blocks. This edition adds two novelties: the vulnerability ladder V1.0–V1.7 (which weakens only the implementation while keeping the digest intact, culminating in algebraic and SMT-based recovery attacks) and the compression of the method v2.0–v2.4 (up to -58% memory and -41% lookups), together with a chapter devoted to practical applications in hardware wallets, compute-in-memory and FPGAs.

2. Background: the original SHA-256 (FIPS 180-4)

SHA-256 is a cryptographic hash function $f : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ defined over 512-bit blocks. It processes each block with 64 identical rounds over eight 32-bit registers. Its constants are not arbitrary: H_0 comes from the fractional parts of the square roots of the first eight primes, and the 64 round constants K_t from the cube roots of the first 64 primes — a choice designed to leave the designer “nothing up his sleeve. Padding guarantees that every message fits an integer number of blocks: a 1 bit, the necessary zeros, and the original bit length as a 64-bit big-endian integer are appended. The schedule W expands the 16 block words to 64 through a nonlinear recurrence, and each round mixes the state with T1 and T2. The output is the sum (modulo 2^{32}) of the initial state and the state after 64 rounds. Security rests on preimage resistance (2^{256}) and collision resistance (2^{128}).

```
M' = M || 1 || 0^k || len64(M)    with k minimal such that (|M|+1+k+64) = 0 (mod 512)
W_t = M_t (t=0..15); W_t = sig1(W_{t-2}) + W_{t-7} + sig0(W_{t-15}) + W_{t-16} (t=16..63)
T1 = h + SIG1(e) + Ch(e,f,g) + K_t + W_t    T2 = SIG0(a) + Maj(a,b,c)
h<-g  g<-f  f<-e  e<-d+T1  d<-c  c<-b  b<-a  a<-T1+T2    (sums mod 2^32)
```

3. The LUT-SHA256 method: theory

The foundational observation is that every finite function IS its table; the difficulty is never conceptual but one of size. LUT-SHA256 solves the size problem by decomposing SHA-256 into byte-local primitives whose tables fit in memory (the largest has 131,072 entries). The five theorems below formalize that decomposition and its exact equivalence with the original method.

Theorem 1 — Functional decomposition If $f = g \circ h$ and both g and h admit finite table representations, then f is computable exclusively by table lookups, using data only as indices. Proof: immediate from the definition of a function as its graph.

Theorem 2 — Byte-locality Every primitive of SHA-256 is a composition of byte-local functions, except the carry chain of addition and the byte recombination of rotations, solved by chaining (Theorems 3 and 4).

Theorem 3 — TADDC carry chain Let $\text{TADDC}(a, b, c) = (s, c')$ with $s = (a + b + c) \bmod 256$ and $c' = \lfloor (a + b + c)/256 \rfloor$. The recurrence $(s_i, c_{i+1}) = \text{TADDC}(w_i, v_i, c_i)$, with $c_0 = 0$ and c_4 discarded, yields exactly $\text{val}(w) + \text{val}(v) \bmod 2^{32}$. Proof by induction on byte i : let P_i be the partial sum of the first $i + 1$ bytes. Base case: $c_1 = \lfloor (w_0 + v_0)/256 \rfloor$. Inductive step: if $c_i = \lfloor P_{i-1}/256^i \rfloor$, then $\lfloor P_i/256^i \rfloor = c_i + w_i + v_i$, whose division by 256 yields exactly c_{i+1} and the residue s_i . Discarding c_4 performs the reduction modulo $256^4 = 2^{32}$. $\square$

Theorem 4 — Byte-recombination rotations For $n = 8k + r$ with $0 \leq r < 8$: $\text{ROTR}^n(w)[i] = (w[i + k \bmod 4] \gg r) | ((w[i + k + 1 \bmod 4] \ll (8 - r)) \& 0xFF)$, and SHR uses the same expression with $w[j] = 0$ for $j \geq 4$. Proof by bit positions: output byte i covers original bits $8i + n$ to $8i + n + 7$, namely bits $r..7$ of byte $i + k$ (placed at positions $0..7 - r$ by the right shift) and bits $0..r - 1$ of byte $i + k + 1$ (placed at positions $8 - r..7$ by the left shift). $\square$

Theorem 5 — Equivalence $\text{LUT-SHA256}(M) = \text{SHA-256}(M)$ for every message M . Proof: structural induction over the classical algorithm; every primitive substitution preserves values exactly, and composition of exact functions is exact. $\square$

The base table inventory (v1.0): TXOR, TAND and TOR (65,536 entries each), TNOT (256), TS and TC (131,072 each, the two TADDC planes) and TL/TR (18×256 for displacements). In total $\approx 402,400$ one-byte entries, at a cost of $\approx 14,272$ lookups per 512-bit block. The inviolable rule of the method: data is never operated upon — only indexed; arithmetic exists solely in the table construction, which happens once and can be audited in isolation.

4. Comparison with the original method

The side-by-side comparison shows that both resolutions produce the same intermediate values and the same digest; the medium changes, not the result. In round 0 of Exercise A (entropy *7fimes16*), the original method computes T1 with 32-bit arithmetic; LUT-SHA256 obtains the same value through four chained carry lookups.

ORIGINAL: T1 = 5be0cd19 + 3587272b + 1f85c98c + 428a2f98 + 7f7f7f7f = 72f76ce7
 LUT: TS[68][7f][0]=e7 TC=0 | TS[ed][7f][0]=6c TC=1 |
 TS[77][7f][1]=f7 TC=0 | TS[f3][7f][0]=72 TC=1 -> 72f76ce7
 SIG1, Ch, T2, a and e match byte by byte across all 64 rounds (Annexes A and B).

Feature	Original method	LUT-SHA256
Data operations	rotations, XOR, AND, OR, sums	none: indexing only
Processing unit	32-bit word	8-bit byte
Addition	32-bit adder	4 chained TADDC lookups
Rotation	hardware/software operator	byte recombination (TROT)
Persistent state	none	8 tables (0.4 MB in v1.0)
Lookups per block	0	14,272 (v1.0) -> 8,361 (v2.4)
Digest	SHA-256	identical (Theorem 5)

5. Experimental verification

All versions of the method (v1.0 and compressions v2.x) were verified against the reference (hashlib, itself validated against FIPS 180-4) in the scenarios below. The digests are identical in all cases, including the

padding boundaries — historically the most common source of bugs in hash implementations — and the zero blocks, which exercise the v2.3 master-table path.

Scenario	Input	Digest (LUT = SHA-256)
Exercise (BIP39)	A 7f x 16 bytes	87dcde7f a6df23e1 5fa7ba9b 2a1f3140 8eac832f 4e615ea8 15ae9202 4e3d818b
Exercise (BIP39)	B 00 x 64 bytes	f5a5fd42 d16a2030 2798ef6e d309979b 43003d23 20d9f0e8 ea9831a9 2759fb4b
FIPS 180-4 vector	"abc"	ba7816bf 8f01cfea 414140de 5dae2223 b00361a3 96177a9c b410ff61 f20015ad
FIPS 180-4 vector	empty	e3b0c442 98fc1c14 9afb4c8 996fb924 27ae41e4 649b934c a495991b 7852b855
Block boundaries	55 / 56 / 57 bytes	match
Long messages	1,000 bytes and random	match
Zero blocks	00x128, 00x256	match

6. Implementation vulnerabilities

By Theorem 5, the cryptographic security does not change in any variant: preimage 2^{256} and collision 2^{128} remain intact. The vulnerabilities of the method are exclusively of implementation, and they deserve precise cataloguing because they define the safe deployment conditions.

- **V1 — Cache side channel (the most serious).** Every lookup uses secret data as an index; the memory access pattern (cache lines, timings) depends on the processed content. This is the exact analogue of the Bernstein and Osvik–Shamir–Tromer attacks against AES T-tables. Mitigation: constant-time access (read all cells), tables in private ROM, or tabulated 1-bit adders (8 entries).
- **V2 — Integrity surface.** The tables define the ENTIRE hash; one corrupted byte silently alters all future digests, and an adversary could “cook” tables with artificial collisions. Mitigation: runtime regeneration from the specification and checksum verification.
- **V3 — Silent bit-flip failures** in a cell. Mitigation: ECC memory and double reads.
- **V4 — Preprocessing host (padding).** Risk identical to the original method; not aggravated.

7. Vulnerability ladder V1.0–V1.7 and attacks

The ladder is an adversarial engineering experiment: each rung weakens ONLY the implementation — the digest remains identical to SHA-256 — and we measure how easily an attacker recovers the original value. Thresholds were quantified with the SMT solver Z3 on a mini variant (8-bit registers, same structure).

Rung	Planted leak	Attack result
V1.0	none	intractable (preimage 2^{256})
V1.1	full lookup trace	trivial: the trace IS the data
V1.2	carry bits only	2,400 bits leaked/block; Z3 fails
V1.4	bus T1 (one sum per round)	pure algebra: 64 steps, no brute force
V1.5	minimal T1 (16 rounds)	64 bytes suffice; digest unused
V1.6	subthreshold T1 (15) + digest	digest completes the gap (Z3: 68.7 s)
V1.7	checkpoints every k rounds	k=2,4 instant; k=8 by climbing (49.6 s); k=16 = wall

The algebraic attack of rung V1.4 is a direct consequence of the two-sequence recurrence: given two consecutive states, $T1_t = e_{t+1} - d_t$ and $W_t = T1_t - h_t - \Sigma_1(e_t) - \text{Ch}(e_t, f_t, g_t) - K_t$. With 16 leaked T1 values the message is recovered exactly, on the real SHA-256, without using the digest. The engineering lesson is blunt: a single observable bus turns preimage into algebra; that is why production implementations must be constant-time and must not leave any intermediate state observable (nor debug registers with checkpoints, as rung V1.7 shows).

8. Compression of the method v2.0–v2.4

Once equivalence is proven, the method admits systematic compression. Every step was verified digest-identical against hashlib before being accepted; the only bug of the process (packing the carry as $s|256$ instead of $s|(c \ll 8)$) was caught with exhaustive tests over the 131,072 TADDC combinations and documented as a warning of how delicate table fusion is.

- **v2.0** — TADDC fusion (sum and carry in a single entry) and triangular storage of symmetric tables (access $t[\text{max}][\text{min}]$): memory 402,400 $\rightarrow$ 169,344 entries (–58%), additions at 1 lookup per byte.
- **v2.1** — TNAD fusion ($\neg e \wedge g$ in one table): Ch from 16 to 12 lookups; 234,880 entries, 11,616 lookups/block. This is the *memory* mode.
- **v2.2** — Macro-rounds $\times 4$: 16 visible steps instead of 64; the macro table matches the reduction phase (macro 0 $\rightarrow$ a=a9d519f4, e=281e9564).
- **v2.3** — Zero-block master table: if the block is all zeros, $W \equiv 0$ is known a priori and the entire schedule and the per-round W_t addend are omitted ($\sim 2,000$ lookups saved per padding block). Crucial for PBKDF2/BIP39.
- **v2.4** — TROT fusion $\text{TROT}[r][x][y] = (x \gg r) | ((y \ll (8 - r)) \& 0\text{xFF})$: rotations at 1 lookup per byte; 655,872 entries, 8,361 lookups/block (–41%). This is the *speed* mode.

Version	Change	Memory (entries)	Lookups/block	Digest
v1.0	base	402,400	14,272	= SHA-256 OK
v2.0	TADDC + triangular	169,344 (–58%)	—	OK
v2.1	+ TNAD	234,880 (–42%)	11,616 (–19%)	OK
v2.2	+ macro $\times 4$	same	16 visible steps	OK
v2.3	+ zero blocks	same	–2,000 per zero block	OK
v2.4	+ TROT	655,872 (speed mode)	8,361 (–41%)	OK

9. Applications and use cases

Beyond the theoretical interest, LUT-SHA256 has a very concrete hardware and auditing profile. This section details the scenarios where its operator-free architecture is a real advantage — and the scenarios where it is not.

9.1. 9.1 Hardware wallets and secure elements (BIP39/PBKDF2)

The BIP39 ecosystem uses SHA-256 at two critical points: the 4–8-bit checksum that validates the mnemonic and, above all, the PBKDF2-HMAC-SHA512 seed derivation, which executes 2,048 compression iterations dominated by zero padding. A hardware wallet could host the tables in certified ROM — a single audit surface verified once and valid for every execution — and a minimal sequencer with no arithmetic unit. The v2.3 variant (zero-block master) trims precisely the padding work that dominates PBKDF2, and the

absence of operators shrinks the logic-failure surface: there are no 32-bit carry chains to fail silently, only ECC-protected memory reads.

9.2. 9.2 Compute-in-Memory (CIM)

In CIM arrays the native physical operation IS the lookup: every memory row is an analog accumulator. A hash expressed entirely as table accesses fits naturally in that technology, without moving data between ALU and memory. The access pattern — deterministic, read-only, write-free — is the ideal case for read-only CIM arrays, and the energy per lookup is below that of an equivalent arithmetic operation in advanced nodes.

9.3. 9.3 FPGAs and low-power ASICs

Modern FPGAs expose several megabits of BRAM and relatively little logic; classical SHA-256 consumes logic LUTs and its critical path runs through the carry chains. LUT-SHA256 inverts the ratio: the tables map 1:1 onto BRAM with decoders, and the critical path reduces to the memory read latency. For low-power ASICs (IoT, security sensors), the design allows a chip without an ALU: table ROM + controller + ECC. The memory mode (v2.1, 235 KB of tables) fits in a single mid-range FPGA BRAM block.

9.4. 9.4 Side-channel research and education

The ladder V1.0–V1.7 is a reproducible attack laboratory: every rung is an exercise with a verifiable solution, from the trivial trace attack (V1.1) to the algebraic inversion of the T1 bus (V1.4) and the checkpoint climbing (V1.7). The Z3 quantification turns qualitative notions ("tables leak") into measurable thresholds (15 of 16 T1 leaks; 32 of 64 T2 leaks; carries never suffice). It is directly reusable material for applied cryptography courses and CTF competitions.

9.5. 9.5 Formal verification and auditing

Equivalence with the standard is guaranteed by Theorem 5, and the polynomial (ANF) rewriting of the method performed during the research connects with formal verification tools: every output bit is an explicit Boolean polynomial over the input bits. For a Common Criteria evaluator, the method offers an attractive property: the correctness of the ENTIRE hash is audited by inspecting the tables (a finite object under 700 KB) and the sequencer, without reasoning about 32-bit arithmetic.

9.6. 9.6 Education

Pedagogically, LUT-SHA256 turns an abstract statement — "every hash is a finite function" — into an executable construction: students can build SHA-256 without knowing how to add, following the byte-wise functional decomposition. The 8-bit mini-variant used in the SMT experiments is ideal as a laboratory exercise.

9.7. 9.7 Platform fit and non-applicable scenarios

The method is neither a mining accelerator nor a throughput improvement on GPUs: uncoalesced, data-dependent accesses destroy memory-level parallelism, and a GPU will run classical vectorized SHA-256 much faster. It must also not be used in production without constant-time countermeasures, due to vulnerability V1. The table below summarizes the platform fit.

Platform	Fit	Reason
CPU (software)	Neutral	equivalent; requires constant time
GPU	Unfavorable	uncoalesced accesses
FPGA (BRAM)	Very favorable	tables 1:1 in BRAM, no carry chains
Low-power ASIC	Very favorable	no ALU: ROM + sequencer + ECC
Compute-in-Memory	Optimal	the lookup IS the physical operation
Secure element / wallet	Favorable	certified ROM, single audit surface
Education / CTF	Favorable	reproducible side-channel lab

10. Conclusions and future work

LUT-SHA256 proves that SHA-256 admits an execution without arithmetic on the data, formally equivalent to the standard (Theorems 1–5), compressible in memory and lookup count (v2.0–v2.4) and precisely analysable with respect to its side-channel surface (ladder V1.0–V1.7). The transversal lesson is twofold: the security of a hash lives both in its mathematics — which here remains intact — and in the discipline of its implementation, and the boundary between the two can be measured, not just intuited. **Future work:** constant-time access (masked reads of all cells), mechanical verification of the theorems in Coq/Isabelle, cache-attack simulation against the ladder rungs, an FPGA prototype comparing BRAM versus LUT implementations, and extension of the v2.3 master table to other padding patterns common in Merkle trees.

11. Annex A — Complete Exercise A (entropy 7f x 16, BIP39)

```

7f7f7f7f 7f7f7f7f 7f7f7f7f 7f7f7f7f 80000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000080
ae4e4e4d ae9e4e4d 9d133fca 7f44648e 783b59e4 bec92153 c7d0be4c 62fa6942
f77c5d8a 16a50382 248e5ed4 99f03aaa a382d694 e1f72214 94ca8e57 89f32ddd
f8d4fb20 100e9cef 24036c4b d243a844 2ec8cc2c a3989a6e e4262dfb d68596d0
211ff173 ef9e1f08 4618fbc4 1daa0c30 a71ed16e 2bdbd9d6 7f5e479b a3114e57
4eae2ca4 2ecc1cbd 8f80444c 75b47f19 74e066c2 339c4c69 a5a5559d f32625ad
dc2ee19b 143c30db 49192bfb aef4a8cf eaa4f03b 3bca7aac 41567214 bbaf225a

```

t	W_t	T1	T2	a	e	t	W_t	T1	T2	a	e
0	7f7f7f7f	72f76ce7	08909ae5	6a09e667	510e527f	1	7f7f7f7f	8733ee50	7ba89322	7b8807cc	18476221
2	7f7f7f7f	c27a74d1	63ab8619	02dc8172	c3a2e1c2	3	7f7f7f7f	be14aefd	ebc06af7	2625faea	7de22356
4	80000000	f0b051d5	d48d7464	a9d519f4	281e9564	5	00000000	051fde04	5ccf9a6c	c53dc639	6c3859a1
6	00000000	054d350f	48166950	61ef7870	07fc5f76	7	00000000	bcd2b1f8	f54bd037	4d639e5f	2b732ff9
8	00000000	07aee218	e9622936	b21e822f	66a7cbec	9	00000000	bd955231	93327bee	f1110b4e	ccca851
10	00000000	ce84f240	ac0a9309	50c7ce1f	1f84caa1	11	00000000	36fd834d	bb84a013	7a8f8549	1be8909f
12	00000000	316fbc43	a1b2264c	f2822360	e91c057c	13	00000000	1096296b	49a884a9	d321e48f	2280c991
14	00000000	d5f7f28a	6eb7d11c	5a3eae14	615df78a	15	00000000	0151db8b	8547330b	44afc3a6	508777d3
16	ae4e4e4d	a14839df	f7e7bc0d	86990e96	f3d3feeb	17	ae9e4e4d	b1921f58	bb84e406	992ff5ec	746a1e6e
18	9d133fca	e368237b	a543f90e	6d3a6d5e	0bd0cd6c	19	7f44648e	5f14b0a7	bf43e22c	88ac1c89	2817e721
20	783b59e4	0bda29db	3f7db6e4	1e589d33	e5adb3fd	21	bec92153	259bdd7c	9309f058	4b57e0bf	a50a1fc7
22	c7d0be4c	8c017ddb	f211a590	b8a5cdd4	92d64ada	23	62fa6942	fb44d2ec	026a77ba	7e13236b	14ad9a64
24	f77c5d8a	4ccb72bd	541df00b	fdaf4aa6	199d65bf	25	16a50382	fa951570	989fe064	a0e962c8	9823537c
26	248e5ed4	69119787	0b6c5962	9334f5d4	b33ae344	27	99f03aaa	35a8128b	de156acc	747df0e9	e724baf2
28	a382d694	6ef89f7d	eee094c5	13bd7d57	33575d31	29	e1f72214	fbba620	a7b31ef1	5dd93442	0fe20245
30	94ca8e57	9984e99a	9f3f01b2	a372c511	8ef49bf4	31	89f32ddd	3e88d9ef	7f4e2952	38c44b4c	0e02da83
32	f8d4fb20	91c9d55a	e34b6be0	bdd70341	52465746	33	100e9cef	a0c8e352	ff675478	7515413a	efa3099c
34	24036c4b	72466c44	8b9b2a3d	a03037ca	443ba863	35	d243a844	f2503886	425ea4e5	fde19681	ab0ab790
36	2ec8cc2c	dd3c788c	51a856c9	34aeddbb	b0273bc7	37	a3989a6e	20a97d5b	df11ef89	2ee4cf55	5251b9c6
38	e4262dfb	e0cd5981	f4298281	ffb6ce4	c0d9b525	39	d68596d0	36522851	8d51e829	d4f6dc02	deae0032
40	211ff173	a7adf3de	7b354e92	c3a4107a	6b0105bc	41	ef9e1f08	c304702e	a8ee60ff	22e34c70	d692cd33
42	4618fbc4	896266e2	7cb5aeaa	6bf2d12d	c2bfdd12	43	1daa0c30	f4070cae	f2a35b17	061815cc	5e5942e4
44	a71ed16e	77a91438	54d51105	e6aa67c5	b7ab1d28	45	2bdbd9d6	8d56f3bb	689f435c	cc7e253d	9a8c60a8
46	7f5e479b	0a889403	821ca4b8	f5f63717	f949c4e8	47	a3114e57	5b283e11	7f0dfaf7	8ca538bb	10a0a9cf
48	4eae2ca4	c606cc33	46f7818f	da363885	41d2a5d6	49	2ecc1cbd	f50ef29e	a0cf3534	0cfe4dc2	9284f170
50	8f80444c	adc5b9cf	807a9b14	95de27d2	eb0529b5	51	75b47f19	e393a71d	7ab93145	2e4054e3	3a6af28a
52	74e066c2	551da697	862ca1e9	5e4cd862	bdc9dfa2	53	339c4c69	8941f708	ba3ea401	bd4a4880	621bf459
54	a5a5559d	01a6f165	e60ef7c8	43809b09	f201eda	55	f32625ad	82707181	aaa7b983	e7b5e92d	2fe74648
56	dc2ee19b	dfb9c2b8	9b5c1dc1	2d18b04	e0bd49c3	57	143c30db	68e76974	f9bd2d8a	735e079	bb240b38
58	49192bfb	338447d8	28366b8d	62a496fe	ace6807d	59	aef4a8cf	6e7c0c0e	a2be96df	5bbab365	1b3a3105
60	aaa4f03b	7726d3f9	0da8680d	113aa2e5	9b94370a	61	3bca7aac	93862159	8fb2a5d0	84cf3c06	725cb472
62	41567214	57a142b7	93d632a5	2338c729	f62ab857	63	bbaf225a	2c638dc	f16f6a4d	eb77755c	b35bf61c

```

def wadd(t, a, b):                                # 32-bit addition via the TADDC chain
    out = [0]*4; c = 0
    for i in range(4):
        v = t.taddc[max(a[i],b[i])][min(a[i],b[i])][c]
        out[i] = v & 0xFF; c = (v >> 8) & 1
    return out
def wrot(t, w, n):                                # ROTR^n via byte recombination
    k, r = divmod(n, 8)
    if r == 0: return [w[(i+k)%4] for i in range(4)]
    return [t.trot[r-1][w[(i+k)%4]][w[(i+k+1)%4]] for i in range(4)]
# inviolable engine rule: data is only indexed;
# arithmetic exists only inside build_tables(), which runs once.

```

14. Annex D — Complete constants

H0: 6a09e667 bb67ae85 3c6ef372 a54ff53a 510e527f 9b05688c 1f83d9ab 5be0cd19

428a2f98 71374491 b5c0fbcf e9b5dba5 3956c25b 59f111f1 923f82a4 ab1c5ed5

d807aa98 12835b01 243185be 550c7dc3 72be5d74 80deb1fe 9bdc06a7 c19bf174

e49b69c1 efbe4786 0fc19dc6 240ca1cc 2de92c6f 4a7484aa 5cb0a9dc 76f988da

983e5152 a831c66d b00327c8 bf597fc7 c6e00bf3 d5a79147 06ca6351 14292967

27b70a85 2e1b2138 4d2c6dfc 53380d13 650a7354 766a0abb 81c2c92e 92722c85

a2bfe8a1 a81a664b c24b8b70 c76c51a3 d192e819 d6990624 f40e3585 106aa070

19a4c116 1e376c08 2748774c 34b0bcb5 391c0cb3 4ed8aa4a 5b9cca4f 682e6ff3

748f82ee 78a5636f 84c87814 8cc70208 90befffa a4506ceb bef9a3f7 c67178f2